

Sistema de Autocomplete de Jogos com Trie

Elainne Rohs

Joana Fontes

Trabalho 4 — Estrutura de Dados

1 Representação da Trie

1.1 Tamanho do alfabeto

Adotamos um alfabeto de **36 caracteres** (`ALPHABET_SIZE = 36`): as 26 letras minúsculas (`a–z`) e os 10 algarismos (`0–9`). O enunciado garante que os títulos contêm apenas letras, números e espaços; como a chave de busca elimina os espaços, esse conjunto cobre exatamente os caracteres que chegam à Trie. Excluimos acentos e símbolos especiais porque não ocorrem nos títulos dos testes e apenas inflacionariam cada nó com entradas `nullptr` desnecessárias.

1.2 Mapeamento caractere $\rightarrow$ índice

A função auxiliar `getCharIndex(c)` realiza o mapeamento direto pela tabela ASCII, garantindo acesso $O(1)$ ao filho correspondente:

letras: 'a' $\rightarrow$ 0, 'b' $\rightarrow$ 1, ..., 'z' $\rightarrow$ 25	(via <code>c - 'a'</code>)
digitos: '0' $\rightarrow$ 26, '1' $\rightarrow$ 27, ..., '9' $\rightarrow$ 35	(via <code>c - '0' + 26</code>)

Cada nó possui um array `children[36]`; como o mapeamento é bijetivo dentro do alfabeto escolhido, não há colisões.

1.3 Conversão para chave de busca (`toSearchKey`)

Antes de qualquer inserção ou consulta, títulos e prefixos passam por `toSearchKey`. O método percorre o texto caractere a caractere e:

- (i) **ignora espaços**;
- (ii) **converte maiúsculas em minúsculas** (deslocamento ASCII constante `'a' - 'A' = 32`);
- (iii) **preserva letras minúsculas e dígitos**;
- (iv) descarta qualquer outro símbolo.

Com isso, "Half Life", "half1" e "HALF L" produzem a mesma chave interna `half1life`, tornando a busca *case-insensitive* e indiferente a espaços. Exemplos de equivalência:

<code>toSearchKey("Hades")</code>	<code>== toSearchKey("hades")</code>	<code>// "hades"</code>
<code>toSearchKey("Half Life")</code>	<code>== toSearchKey("half1life")</code>	<code>// "half1life"</code>
<code>toSearchKey("Portal 2")</code>		<code>// "portal2"</code>

2 Funcionamento dos Métodos

2.1 `insert(Game* game)`

Calcula a chave do título via `toSearchKey` e, partindo da raiz, desce na Trie caractere a caractere. Quando o filho correspondente não existe, um novo nó é alocado; caso exista, simplesmente desce. Ao consumir toda a chave, marca o nó corrente com `isEndOfTitle = true` e armazena o ponteiro para o objeto `Game` já existente no array — a Trie **nunca cria cópias** dos jogos.

2.2 `contains(string title)`

Converte o título para chave e desce na Trie seguindo cada caractere. Se em qualquer ponto o filho for `nullptr`, retorna `false` imediatamente. Ao atingir o nó final da chave, retorna o valor de `isEndOfTitle` — garantindo que um prefixo válido mas não registrado como título completo (ex: "hal") seja corretamente identificado como ausente.

2.3 `autocomplete(string prefix, int k)`

Se $k \leq 0$, retorna vetor vazio imediatamente. Caso contrário, o método executa quatro etapas:

1. **Localização do prefixo:** converte o prefixo para chave e desce na Trie. Se o caminho não existir, retorna vazio.
2. **Coleta (DFS):** a partir do nó final do prefixo, a função auxiliar `collectGames` realiza uma busca em profundidade pela subárvore, acumulando ponteiros de todos os nós com `isEndOfTitle = true`. O array original de jogos **nunca é percorrido** nesta etapa.
3. **Ordenação:** `sortResults` ordena o vetor resultante pelos critérios definidos.
4. **Truncamento:** o vetor é reduzido a no máximo k elementos antes de ser retornado.

2.4 sortResults(vector<Game*>& games)

Implementa **Insertion Sort** manual (sem `std::sort`) com dois critérios em cascata: (1) maior popularidade; (2) em caso de empate, ordem alfabética da chave de busca do título. A complexidade é $O(r^2 \cdot L)$, onde r é o número de resultados e L o custo de recalculer a chave em cada comparação.

3 Análise de Custo Computacional

Parâmetros: L = comprimento da chave do título; p = comprimento da chave do prefixo; s = nós visitados na subárvore do prefixo; r = jogos encontrados; A = tamanho do alfabeto (36, constante).

Tabela 1: Complexidade das operações da Trie

Operação	Custo	Origem do custo
<code>insert</code>	$O(L)$	Percorre os L caracteres da chave; cada passo cria um nó em $O(A)$ constante ($A = 36$).
<code>contains</code>	$O(L)$	Desce L níveis com acesso $O(1)$ ao filho em cada passo.
<code>autocomplete</code>	$O(p + s \cdot A + r^2 \cdot L)$	$O(p)$ para localizar o prefixo; $O(s \cdot A)$ na DFS (cada nó inspeciona A filhos); $O(r^2 \cdot L)$ no Insertion Sort — $O(r^2)$ comparações, cada uma recalcula <code>toSearchKey</code> em $O(L)$.

Como A é constante (36), `insert` e `contains` são lineares no comprimento da chave e **independentes de N** . O custo dominante do `autocomplete` para catálogos grandes é a DFS: $O(s \cdot A)$, onde s cresce apenas com o número de nós sob o prefixo — e não com N .

4 Comparação com Solução Ingênua e Uso de Memória

4.1 Comparação de desempenho

Para catálogos grandes com consultas frequentes, a Trie é decisivamente mais rápida: enquanto a busca ingênua cresce com N , a Trie mantém $O(p + s)$ após a construção.

4.2 Custo de memória

A solução ingênua guarda apenas o array de N objetos `Game`. A Trie acrescenta uma árvore de nós: no pior caso (sem prefixos compartilhados) o número de nós é $O(N \cdot L)$. Cada nó carrega um array de 36 ponteiros (288 bytes em 64 bits), um `bool` e um ponteiro para `Game`, totalizando aproximadamente 300 bytes por nó.

Tabela 2: Comparação entre a Solução Ingênua e a Estrutura em Trie

Critério	Solução Ingênua (array)	Trie
Busca por prefixo	$O(N \cdot L)$ — varre todo o catálogo	$O(p + s)$ — segue só o caminho do prefixo
Escalabilidade	Degrada com N grande	Independente de N após construção
Construção	Nenhuma (apenas o array)	$O(N \cdot L)$ — insere cada título
Memória	$O(N)$ objetos <code>Game</code>	$O(N \cdot L)$ nós + array [36] por nó
Prefixos comuns	Sem benefício	Nós compartilhados — menos memória

Há, portanto, uma troca clássica tempo-espço: paga-se mais memória para ganhar velocidade de busca por prefixo.

4.3 Compartilhamento de prefixos

Títulos com início comum dividem nós: "Halo", "Half Life" e "Hades" compartilham os nós de "ha", criando-os uma única vez. Quanto mais prefixos em comum, menor o número total de nós e menor o consumo de memória em relação ao pior caso.

4.4 Efeito do tamanho do alfabeto

Como cada nó reserva um array fixo de A ponteiros, o espaço por nó cresce linearmente com A . Um alfabeto maior (incluindo acentos ou símbolos) desperdiçaria memória com muitas entradas `nullptr`. O alfabeto enxuto de 36 posições mantém os nós compactos sem perder nenhum caractere efetivamente utilizado nos títulos.